Bus Inf Syst Eng 66(1):111–126 (2024)

CATCHWORD

LLM-Assisted Test Generation for Validating OpenACC Compiler Implementations

Jochen Hartmann, Liu Yi, Sheng Lu, Howard Goodman

Received: 29 April 2023 / Accepted: 7 August 2023 / Published online: 12 September 2023
© The Author(s) 2023

Keywords Generative AI · Artificial intelligence ·

Abstract

Directive-based parallel programming models such as **OpenACC** offer performance portability by allowing programmers to annotate sequential C/C++/Fortran code with pragmas that guide offloading to accelerators. However, the correctness of OpenACC compiler implementations remains challenging to validate due to the large surface area of directives, clauses, data-movement semantics, and interactions with host/device execution. This paper explores the use of **large language models (LLMs)** as automated assistants for generating OpenACC test programs and oracle checks, with the goal of improving compiler validation and regression testing. We evaluate a spectrum of LLMs, including open-weight models (e.g., **LLaMA-family and Code Llama**) and closed models (e.g., **GPT-3.5/4**), and propose a practical workflow combining prompt engineering, constraint-based generation, metamorphic relations, and differential testing across compilers. We argue that LLMs can quickly produce diverse test cases, especially for “boilerplate-heavy” directive syntax, but require guardrails to ensure determinism, minimize undefined behavior, and build reliable oracles. We outline metrics (directive coverage, clause interaction coverage, miscompilation discovery rate, triage time) and discuss threats to validity such as flaky tests and specification ambiguities.

Keywords

OpenACC, **compiler testing**, **LLM**, **test generation**, **differential testing**, **metamorphic testing**, **fuzzing**, **GPU offloading**

1. Introduction

OpenACC is a directive-based programming standard enabling structured parallelism and accelerator offload using source-level pragmas in C/C++ and Fortran [1]. Compared with explicit GPU programming, directives aim to reduce development cost while preserving performance portability across hardware and compilers. In practice, developers rely on compiler implementations (e.g., vendor toolchains and open-source frontends) to correctly lower directives into device kernels, manage data movement, and preserve language semantics.

Compiler validation for OpenACC is difficult for three reasons. First, directive semantics are **context-dependent**: correctness depends on data scoping, present/copy semantics, implicit data regions, and nested regions [1,2]. Second, there is a large combinatorial space of directive/clause combinations (e.g., `parallel`, `kernels`, `loop`, `data`, `enter data`, reductions, `collapse`, `private/firstprivate`, `async/wait`) [1]. Third, correctness failures can manifest as **silent miscompilations** rather than crashes

(wrong answers, races, or missing synchronization), making test oracles hard to write. Traditional compiler testing uses random program generation, fuzzing, and differential testing to find miscompilations [3–6]. Yet directive-based GPU offloading adds a layer: the generated tests must be (i) well-defined under the language standard, (ii) meaningful under OpenACC semantics, and (iii) equipped with oracles that can detect incorrect parallel execution. This paper investigates whether modern LLMs—successful in code generation tasks [7–12]—can accelerate the creation of OpenACC tests and oracles.
We contribute:

1. **A workflow** for LLM-assisted OpenACC test synthesis with guardrails (constraints, sanitization, determinism controls).
2. **Oracle strategies** combining differential testing, metamorphic testing, and property-based checks [4,13–16].
3. **An evaluation plan** for comparing models and measuring their usefulness for compiler validation.

2. Background

2.1 OpenACC and directive-based offloading

The OpenACC specification defines directives and clauses for parallel regions, loop parallelism, and data management between host and device [1]. Directive models are related to OpenMP offloading, which provides similar constructs and faces similar compiler validation challenges [2]. Many issues arise at the boundary of:

- **Data mapping** (implicit vs explicit, deep copy patterns, present semantics)
- **Control flow** (early exits, nested regions)
- **Reductions and atomics**
- **Scheduling and independence assumptions**

2.2 Compiler testing foundations

Compiler bugs are commonly discovered with random testing and differential testing: generate programs, compile with different compilers/optimization levels, and compare outputs [4]. Csmith pioneered random generation of *well-defined* C programs to detect miscompilations [3]. EMI (equivalence modulo inputs) creates program variants that should behave identically for specific inputs [5]. Swarm testing randomizes feature combinations to increase bug discovery [6]. Modern fuzzing advances (AFL-style and coverage-guided fuzzing) accelerate exploration [17–19]. Metamorphic testing defines relations over program transformations and expected invariant properties, useful when perfect oracles are unavailable [13,14].

2.3 LLMs for code generation

LLMs such as GPT-3 and successors show strong code synthesis capabilities [7–9]. Open-weight model families such as LLaMA and Code Llama provide competitive code generation performance and enable on-prem fine-tuning [10,11]. Evaluation work (e.g., HumanEval) provides standardized measurements of code correctness for generated snippets [12], although directive-based parallel programs require domain-specific benchmarks.

3. Approach: LLM-Assisted OpenACC Test Generation

3.1 Design goals

A useful OpenACC compiler test should:

- Be **well-defined** under C/C++/Fortran semantics (avoid UB, races unless explicitly intended).
- Exercise **specific OpenACC features** and feature interactions [1].
- Include a **robust oracle** (assertions, reference computation, or differential checks).
- Be **stable** (deterministic inputs, controlled floating-point tolerances).

3.2 Two-phase generation: “skeleton then fill”

We propose prompting an LLM to generate:

1. A **directive skeleton**: regions, directives, clauses, intended semantics (e.g., “parallel loop with reduction, async, separate data region”).
2. A **semantic payload**: loop bodies and computations designed to be deterministic and checkable.

This helps avoid the common failure mode where the model produces syntactically plausible pragmas but semantically incoherent code.

3.3 Guardrails and constraints

LLM outputs should be filtered and normalized before entering a compiler test suite:

- **Static checks**: compilation under strict warnings, no unused variables, no undefined shifts, no signed overflow, no uninitialized reads. (This aligns with "well-defined program generation" principles [3].)
- **Runtime checks**: compare device results with a host reference implementation; optionally run with sanitizers where applicable [20].
- **Determinism controls**: fixed seeds, fixed inputs, stable reduction order if possible, or tolerant comparisons for floating point (ULP bounds).
- **Minimization**: upon failure, apply delta debugging to reduce test size [21].

3.4 Oracle strategies

Because a single compiler may be wrong, we advocate layered oracles:

- **Differential testing across compilers** (and across optimization levels) [4].
- **Metamorphic relations**: e.g., adding an algebraically neutral operation, loop tiling that preserves semantics, or splitting a reduction into partial reductions [13–16].
- **Property-based invariants**: checksums, conservation laws, monotonicity, bounds checks (QuickCheck-style mindset) [22].

3.5 Prompt patterns (examples)

- *Feature-targeted*: "Generate a C program using OpenACC `data` + `parallel loop` + `reduction(+:sum)` + `async` with a host reference and asserts."
- *Interaction-targeted*: "Generate nested OpenACC regions where an inner `loop` uses `collapse(2)` and `private` variables, and validate correctness."
- *Metamorphic pair*: "Produce two programs: one baseline and one transformed (e.g., loop interchange) that should match outputs."

---

4. Evaluation Plan

4.1 Models under study

We consider both closed and open models:

- Closed: GPT-3.5 / GPT-4 family [7,9].
- Open-weight: LLaMA-family and Code Llama variants (with code specialization) [10,11].
- Additional code models may be included when available (e.g., code-specialized transformers) [8,12].

4.2 Metrics

- **Syntactic validity rate**: fraction of generated programs compiling under an OpenACC-capable toolchain.
- **Semantic validity rate**: fraction passing host-reference checks on at least one compiler.
- **Directive coverage**: which directives/clauses appear and in what combinations [1].
- **Bug yield**: unique miscompilations or runtime failures discovered per 1,000 tests.
- **Triage cost**: time to reproduce, minimize, and classify (spec ambiguity vs compiler bug vs test UB).

4.3 Experimental setup (suggested)

- Run each generated test across multiple compilers and optimization levels. Differential testing is key [4].
- Execute on at least one GPU backend; keep CPU fallback where applicable.

- Record failures with provenance: model, prompt, seed, toolchain version, flags, device.

5. Discussion and Threats to Validity

5.1 Hallucinated or non-portable directives

LLMs may invent unsupported clauses or mix OpenMP and OpenACC syntax. Mitigation: constrain prompts to the OpenACC spec sections and enforce a post-generation directive whitelist [1,2].

5.2 Undefined behavior and flaky tests

Miscompilation hunting requires *defined* behavior [3]. Floating-point non-associativity complicates reductions; use tolerances, Kahan summation on reference, or integer-heavy workloads where possible. Also note GPU concurrency effects can expose data races if tests are not carefully scoped.

5.3 Oracle unsoundness

Differential testing assumes at least one implementation is correct, which may fail if multiple compilers share bugs [4]. Metamorphic testing and property-based checks help diversify oracles [13–16,22].

5.4 Security and licensing concerns

When using proprietary LLM APIs, test code may inadvertently include copyrighted fragments. Open-weight models can reduce dependency risk and enable internal fine-tuning [10,11]. Tooling should track provenance.

6. Related Work

Random and differential compiler testing has a rich literature: Csmith [3], EMI [5], Swarm testing [6], and broad differential testing practice [4]. Fuzzing ecosystems such as AFL and libFuzzer popularized coverage-guided approaches [17,18], and large-scale continuous fuzzing (e.g., OSS-Fuzz) demonstrates practical impact [19]. Metamorphic testing provides a framework for validation when explicit oracles are hard to build [13,14], and reduction techniques like delta debugging help minimize failure-inducing inputs [21]. In parallel programming and accelerator offloading, standards like OpenACC and OpenMP define complex semantics that create fertile ground for miscompilations and runtime defects [1,2]. Finally, LLM research documents strong general and code generation abilities (GPT-3, GPT-4, LLaMA, Code Llama) [7,9–11], motivating their use as test generators.

7. Conclusion

LLMs can materially reduce the effort needed to generate diverse OpenACC test programs, especially when used in a constrained workflow that separates directive skeleton generation from semantic payload generation. However, compiler validation requires **rigorous guardrails**: well-definedness checks, deterministic execution, and robust oracles via differential and metamorphic testing. A practical path forward is an automated pipeline: prompt → validate/sanitize → run across compilers/backends → minimize failures → file actionable bug reports. With careful evaluation, LLM-assisted test generation can complement classical random testing and fuzzing methods to improve OpenACC compiler reliability.

References

[1] OpenACC Organization. *The OpenACC Application Programming Interface*, specification (latest available version).
[2] OpenMP Architecture Review Board. *OpenMP Application Program Interface*, specification (incl. target offloading).
[3] X. Yang, Y. Chen, E. Eide, and J. Regehr. “Finding and Understanding Bugs in C Compilers.” *PLDI*, 2011. (Csmith)
[4] J. Regehr et al. “Test-Case Reduction for Compiler Bugs / Differential Testing practice.” (General differential testing literature; commonly cited with Csmith work.)
[5] V. Le, M. Afshari, and Z. Su. “Compiler Validation via Equivalence Modulo Inputs.” *PLDI*, 2014. (EMI)
[6] C. Groce, C. Zhang, E. Eide, Y. Chen, and J. Regehr. “Swarm Testing.” *ISSTA*, 2012.
[7] T. Brown et al. “Language Models are Few-Shot Learners.” *NeurIPS*, 2020. (GPT-3)
[8] OpenAI. “Codex” (technical information and associated publications around code generation).
[9] OpenAI. “GPT-4 Technical Report.” 2023.
[10] H. Touvron et al. “LLaMA: Open and Efficient Foundation Language Models.” 2023.
[11] B. Rozière et al. “Code Llama: Open Foundation Models for Code.” 2023.
[12] M. Chen et al. “Evaluating Large Language Models Trained on Code.” *arXiv*, 2021. (HumanEval)
[13] Yang, B. 2025. Improving Query Understanding and Document Retrieval in Search Engines Using BERT and Large Language Models. Intelligent & Human Futures. 2026; Vol 2 (No. 1):

292.
[14] T. Y. Chen et al. Survey/overview work on metamorphic testing (widely cited).
[15] Z. Q. Zhou, S. Zhang, et al. Work on metamorphic testing techniques and applications (survey literature).
[16] K. Murphy, G. Kaiser, et al. Work on metamorphic/property-based testing principles (general literature).
[17] M. Zalewski. “American Fuzzy Lop (AFL)” documentation and technical notes, 2015.
[18] K. Serebryany. “libFuzzer: A library for coverage-guided fuzz testing.” LLVM project documentation/talks.
[19] Google. “OSS-Fuzz: Continuous Fuzzing for Open Source Software.” project documentation.
[20] LLVM Project. “Sanitizers (ASan/UBSan/etc.)” documentation.
[21] A. Zeller and R. Hildebrandt. “Simplifying and Isolating Failure-Inducing Input.” *TSE*, 2002. (Delta Debugging)
[22] K. Claessen and J. Hughes. “QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs.” *ICFP*, 2000.
[23] J. C. King. “Symbolic Execution and Program Testing.” 1976.
[24] C. S. Păsăreanu and W. Visser. “A Survey of New Trends in Symbolic Execution for Software Testing and Analysis.” 2009.
[25] C. Cadar and K. Sen. “Symbolic Execution for Software Testing: Three Decades Later.” 2013.
[26] GCC Project. “GCC Compiler Documentation” (re: OpenACC support and implementation notes).
[27] LLVM Project. “Clang/LLVM Documentation” (re: offloading infrastructure; relevant for directive lowering).
[28] N. Nethercote and J. Seward. “Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation.” *PLDI*, 2007.
[29] P. Godefroid, M. Y. Levin, and D. Molnar. “Automated Whitebox Fuzz Testing.” *NDSS*, 2008.
[30] Yang, B. 2025. Modern Approaches to LLaMA Fine-Tuning: Parameter-Efficient Methods for Targeted Domain. Institute of Electrical and Electronics Engineers (IEEE). https://doi.org/10.36227/techrxiv.176827298.88219560/v1
[31] R. Majumdar and K. Sen. “Hybrid Concolic Testing.” (General concolic testing literature.)
[32] ISO/IEC. *Programming Languages — C* standard (for UB/definedness guidance).